Section 1

2025年12月9日 星期二 13:25

Digital Electronics & Computer Architecture Lab. S1.
1st December, 2025
Lab. Charles Wu.

Reference: DECA Part2, section 1.

Aim. Learn about memory, build a ROM which stores a message ≤ 15 words inside; and build a message writer with trigger as challenge.

Procedure.

- Before the lab:

Memory allows storage of large blocks of data. One word of data can be accessed by computer at once in the memory as following:

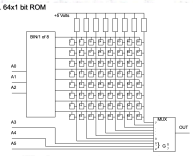


Fig1.1

Before the lab, we need to know about ASCII symbols, and prepare a personalised text message in hexadecimal format.
Mine is: S H A N G H A I.
which is: 0x53 0x48 0x41 0x4E 0x47 0x48 0x41 0x49.
which includes 8 characters in total.

- Initialising ROM

Contents in ROM must be defined when it's created and cannot change at runtime.
We use a synchronous ROM, which requires a rising clock edge to update output to do experiment. I enter the text message "SHANGHAI" as following:

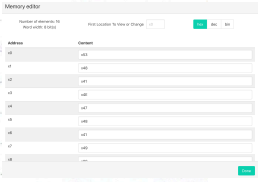


Fig1.2

- Simulating ROM.

After adding address A & output D ports, where the A is 4 bits to select 16 outputs; D is 8 bits, same as ASCII symbol of letters.
∵ ROM is clocked, ∴ it will only update output when it receives a rising clock.
Set A = 2 ; 5 ; 7
Expect to get D = x41 ; x48 ; x49 when clock rises, and the result is as expected:

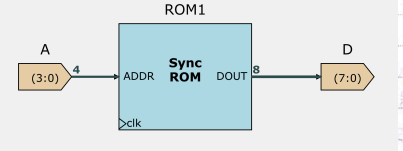
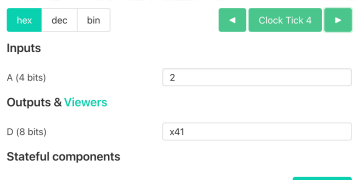
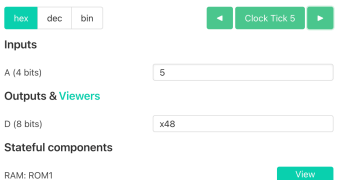
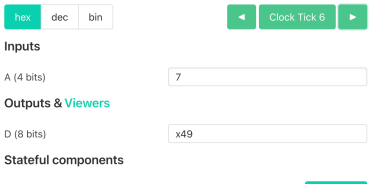


Fig1.3







RAM: ROM1 View

RAM: ROM1 View

Fig1.4, 1.5, 1.6

• Challenge : Message Writer .

We need to connect a counter (in part 1 , section 3) to ROM, so that it will output each character in message sequentially. It will automatically cycle through message with 1-tick delay, as the ROM is synchorous.

After importing my counter, I made the following changes:

① Set C_1 = b0001, so that counter + b0001 each tick

② Set C_2 = 0, which is addition (0 for add ; 1 for substract)

③ Set bus-compare to b0111 (= 7), as there're 8 bits in my message, so counter will cycle from b0000 ~ b0111.

Note : I search online and realise bus-compares are XNOR gates, it will output 0 if A≠B , 1 if A=B , which inputs back to MUX to choose cycle or +1 ;

Also, another notice is that the default input for a no-input component is 0, so my counter will start at 0.

My counter is like :

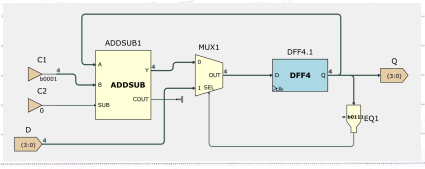


Fig1.7

where D here is a starting point after 1st row.

If set D=n, ROM will go : 0-1-2-3-4-5-6-7-n-(n+1)⋯-7-n because MUX will select n to be next; -(n+1)⋯-7-n ⋯

if output = 7.

Example n=5 : S-H-A-N-G-H-A-I-H-A-I-H-A-I-⋯

If just wants SHANGHAI cycle: set D=n=0.

As expected :

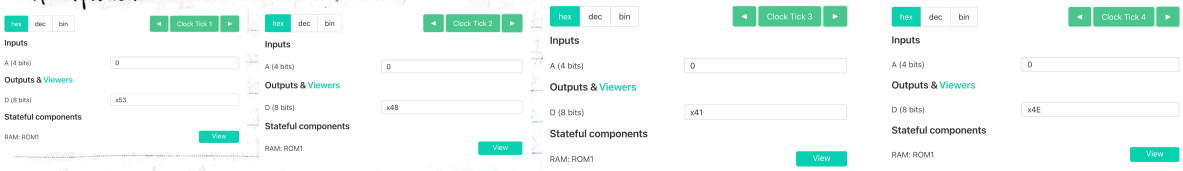


Fig1.8~1.12

Challenge II : adding a trigger.

In this task, we need to add a trigger: if trigger = 1, ROM will run as normal. As long as trigger = 0, ROM will run to end of this cycle, and stop. If trigger = 0 and ROM is at 0, it will never run.

which indicates as table :

Counter	Condition	Next value
Q = 0	T = 0	0
Q = 0	T = 1	Q+1
0 < Q < N-1	Always	Q+1
Q = N-1	Always	0

We now need to add additional MUX and logics to make this design.

My idea is as following. I add a built-in trigger2 into counter to control selection of 0 or +1 in counter. And trigger2 is controlled by logics in big circuit.

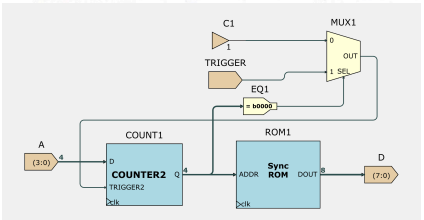


Fig1.13

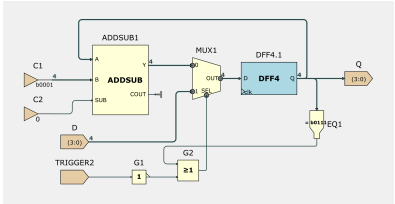


Fig1.14 For counter2

① I consider all conditions that out = 0.

The first is in=0 & trigger = 0
The second is in= N-1, no matter trigger's state.
②. For in=0 & trigger =0, I add a bus-compare to compare in, if in=0, compare inputs 1 into MUX, and Mux outputs trigger value into trigger2. if trigger =1, ∵ there's an OR gate in counter, so (A or 0) always give A, counter runs as normal and will not be affected. If trigger =0, then the OR gate (A or 1)2 will always give D, which means it stays at D.
③. For in≠0, then in big circuit, bus-compare always inputs 0 into MUX and trigger2 is fixed at 1, like former case, counter runs as normal. like before.
Note: we set D=0; so RAM stays at 0 for in=0 & trigger =0.
Test and it's like expected:

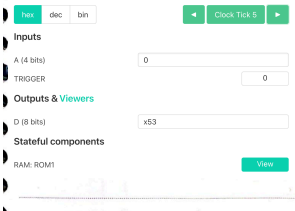


Fig1.15 Remains at 0 when trigger=0

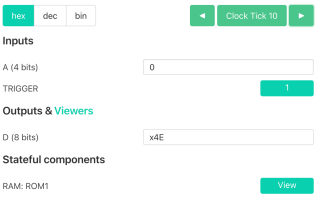
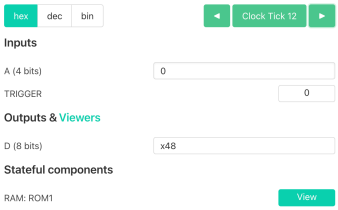


Fig1.16 Run when trigger=1



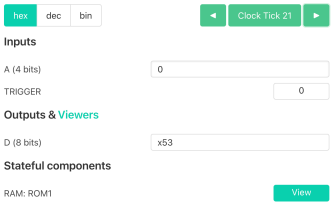


Fig1.17, 1.18 Continue to end when trigger=0 again; stop at 0